

Exercício em duplas: qual é a saída? (1/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual conceito da aula explica?

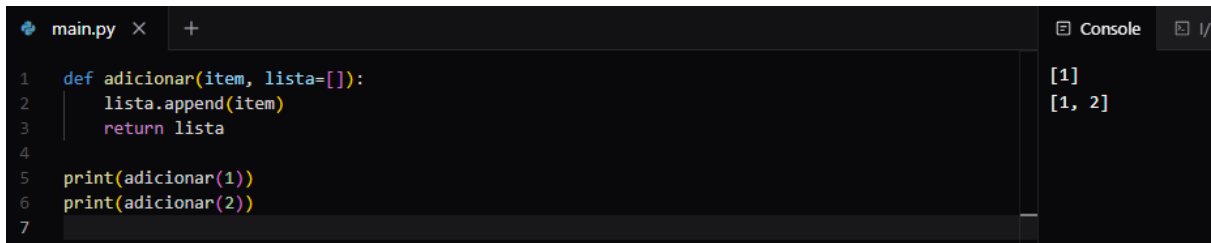
1 · Python

```
def adicionar(item, lista=[]):  
    lista.append(item)  
    return lista  
  
print(adicionar(1))  
print(adicionar(2))
```

2 · Java

```
static void zera(int[] v, int n) {  
    v[0] = 0;  
    n = 0;  
}  
int[] v = {5, 5}; int n = 5;  
zera(v, n);  
System.out.println(v[0] + " " + n);
```

SAÍDAS:



The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'adicionar' that takes an 'item' and a 'lista' (list) as arguments. The 'lista' parameter has a default value of an empty list '[]'. The function appends the 'item' to the 'lista' and returns the 'lista'. Below the function definition, there are two print statements: 'print(adicionar(1))' and 'print(adicionar(2))'. To the right of the code editor, there is a 'Console' window showing the output of the program. The output consists of two lines: '[1]' and '[1, 2]'. The first line corresponds to the first print statement, and the second line corresponds to the second print statement. The console window also has a 'I/O' tab.

```
1 def adicionar(item, lista=[]):  
2     lista.append(item)  
3     return lista  
4  
5 print(adicionar(1))  
6 print(adicionar(2))  
7
```

Console

```
[1]  
[1, 2]
```

```
Main.java × +
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5
6         int[] v = {5, 5}; int n = 5;
7         zera(v, n);
8         System.out.println(v[0] + " " + n);
9
10    }
11
12    static void zera(int[] v, int n) {
13        v[0] = 0;
14        n = 0;
15    }
16 }
```

Console

0 5

Exercício: qual é a saída? (2/3)

Para cada trecho: (a) qual é a saída? (b) qual conceito da aula explica?

3 • Python

```
fs = [lambda: i for i in range(3)]
print([f() for f in fs])
```

4 • C

```
int contador(void) {
    static int n = 0;
    return ++n;
}
// em main:
contador(); contador();
printf("%d\n", contador());
```

Python: [1] e depois [1, 2]. O valor padrão lista=[] é avaliado UMA vez, na DEFINIÇÃO da função, e a mesma lista é reutilizada em todas as chamadas. É um dos bugs mais famosos do Python. Correção: lista=None e, dentro, "if lista is None: lista = []". Conceitos: parâmetros padrão (p. 368) e passagem por atribuição (p. 384).

Java: "0 5". O array é um objeto (a cópia da referência permite alterar v[0]), e o int é copiado. Conceito: Java passa tudo por valor (p. 383).

SAÍDAS:

```
main.py × +
1 fs = [lambda: i for i in range(3)]
2 print([f() for f in fs])
3
```

Console

[2, 2, 2]

```
C Main.c x + Console
1  #include <stdio.h>
2
3
4  int contador(void) {
5      static int n = 0;
6      return ++n;
7  }
8
9  int main()
10 {
11     contador(); contador();
12     printf("%d\n", contador());
13 }
14
15
```

Python: [2, 2, 2]. As lambdas capturam a VARIÁVEL *i*, que no fim vale 2 (late binding).
Correção: lambda i=i: i. Conceito: fechamentos (p. 404), o mesmo problema do var em JS.

C: 3. A variável static mantém o valor entre as chamadas. Conceito: variáveis locais estáticas (p. 373-374).

Exercício: qual é a saída? (3/3)

Para cada trecho: (a) qual é a saída? (b) qual conceito da aula explica?

5 • Rust

```
fn dobra(v: Vec<i32>) -> Vec<i32> {
    v.iter().map(|x| x * 2).collect()
}
let v = vec![1, 2, 3];
let d = dobra(v);
println!("{:?} {:?}", v, d);
```

6 • Python

```
total = 0

def adiciona(x):
    total = total + x
    return total

print(adiciona(5))
```

SAÍDAS:

Erros nas duas

```
HelloWorld.rs x
1 fn dobra(v: Vec<i32>) -> Vec<i32> {
2     v.iter().map(|x| x * 2).collect()
3 }
4
5 fn main() {
6     let v = vec![1, 2, 3];
7     let d = dobra(v.clone());
8     println!("{:?} {:?}", v, d);
9 }
```

Console I/O

[1, 2, 3] [2, 4, 6]

// o código dava erro, então adicionei esse .clone().

```
main.py x +
1 total = 0
2
3 def adiciona(x):
4     return total + x
5
6 print(adiciona(5))
7
```

Console

5

// corrigi também

Rust: NÃO COMPILA (erro E0382). v foi MOVIDO para dobra, que tomou posse do vetor.
Correção: fn dobra(v: &Vec<i32>) e dobra(&v). Conceito: modos de passagem em Rust.

Python: UnboundLocalError. Como há uma atribuição a 'total' dentro da função, o Python decide em tempo de compilação que 'total' é LOCAL, e a leitura acontece antes de haver valor. Correção: "global total" ou, melhor, receber e devolver o valor. Conceito: ambiente de referenciamento local (p. 373) e escopo (aula anterior).